Mazais praktiskais darbs: X-Sudoku ar Simulated Annealing

Kristiāns Rasiņš
kr26040

2026. gada oktobris

1. Uzdevuma formulējums

Izvēlējos X-Sudoku. Tas ir parasts Sudoku, tikai ar vienu papildu noteikumu: abās galvenajās diagonālēs arī katram ciparam jāparādās tieši vienu reizi. Vispārīgi laukums ir $n \times n$, kur $n = b^2$ ($b = 2, 3, 4, 5$, t.i. 4×4, 9×9, 16×16, 25×25), un ir doti daži cipari ("clues"), kurus mainīt nedrīkst. Jāaizpilda pārējās šūnas tā, lai katrā rindā, kolonnā, $b \times b$ blokā un abās diagonālēs būtu visi cipari $1..n$.

Sudoku īstenībā ir apmierināmības uzdevums, nevis optimizācijas, bet to var pārvērst par optimizāciju: skaitām, cik noteikumi ir pārkāpti, un to minimizējam. Optimums ir 0 (atrisināts Sudoku), tāpēc šajā gadījumā "labākais zināmais risinājums" visām instancēm ir zināms jau iepriekš, kas testēšanu padara diezgan ērtu.

2. Algoritma apraksts

Viss ir uzrakstīts Python valodā (3.13), tikai ar standarta bibliotēku, bez gataviem optimizācijas rīkiem. Kods ir mapē `xsudoku/`.

a) Domēns

Risinājums ir viss režģis. Plānošanas mainīgie ir tukšās šūnas, vērtību kopa ir $\{1..n\}$, dotās šūnas ir fiksētas. Sākuma risinājumu taisu tā: katrā blokā ieliek trūkstošos ciparus nejaušā secībā. Tādējādi bloku noteikums ir izpildīts jau no paša sākuma, un, tā kā gājiens (sk. c) arī maina tikai šūnas viena bloka iekšienē, tas paliek izpildīts visu laiku. Tas būtībā samazina meklēšanas telpu un vienu noteikumu no izmaksu funkcijas var izmest.

b) Novērtēšana

Izmaksas ir dublikātu skaits rindās, kolonnās un abās diagonālēs:

$$f = \sum_{u \in \text{rindas, kolonnas, 2 diagonāles}} \big(n - |\{\text{dažādi cipari } u\}|\big) = \sum_{u} \sum_{v=1}^{n} \max(0,\ \text{cnt}_u(v) - 1).$$

Visi noteikumi ir "hard" un vienādi svarīgi, tāpēc atsevišķus svarus neliku. Bloki netiek skaitīti, jo tie vienmēr ir korekti. $f = 0$ tad un tikai tad, ja režģis ir pareizs X-Sudoku. Katram galarezultātam vēl atsevišķi pārbaudu visas $3n + 2$ vienības ar neatkarīgu funkciju (`is_valid_solution`), lai būtu drošs, ka nav kļūda inkrementālajā skaitīšanā.

Izmaksas netiek pārrēķinātas no nulles. Glabāju skaitītājus $\text{cnt}_u(v)$ katrai vienībai, un pēc gājiena izmaiņu (Δf) var izrēķināt $O(1)$ laikā, jo katra šūna pieder ne vairāk kā 4 vienībām. Bez tā Python versija būtu krietni lēnāka.

c) Gājiens

Gājiens ir divu brīvo šūnu vērtību apmaiņa (swap) viena bloka ietvaros. Uztaisīju divus variantus:

- **random** – izvēlas nejaušu bloku (kurā ir vismaz 2 brīvas šūnas) un tajā divas nejaušas brīvas šūnas;
- **conflict** – pirmo šūnu mēģina izvēlēties tādu, kas pašlaik ir konfliktā (tās cipars atkārtojas rindā, kolonnā vai diagonālē; līdz 20 mēģinājumiem), otro ņem nejauši no tā paša bloka.

Sākumā bija tikai *random*, bet uz lielākiem laukumiem tas strādāja diezgan slikti, jo lielākā daļa gājienu aiztiek šūnas, kurās jau viss ir kārtībā. *conflict* variants ir mazliet "mantkārīgāks", bet joprojām ir stohastisks.

d) Algoritms

Simulated Annealing, praktiski tieši pēc lekcijas pseidokoda `SIMULATED_ANNEALING`(φ, f, V, t, L): gājienu ar $\Delta f \leq 0$ pieņem vienmēr, sliktāku pieņem ar varbūtību $e^{-\Delta f / t[k]}$, un paralēli atceras labāko redzēto risinājumu. Parametri:

- sākuma temperatūra t_0 – standartnovirze Δf vērtībām no 200 nejaušiem gājieniem sākuma stāvoklī (testos sanāca no 1.2 līdz 2.8);
- atdzišana ģeometriska: $t[k] = t_0 \cdot \alpha^k$, $\alpha = 0.999$;
- $L[k] =$ brīvo šūnu skaits (konstants visām temperatūrām);
- kad $t < 0.01$ ("$t \approx 0$"), temperatūru uzsilda atpakaļ uz t_0 (reheat) un turpina no esošā stāvokļa. To pievienoju pats, jo citādi pēc pirmās atdzišanas viss beidzas, un uz grūtākajām instancēm tad parasti vēl paliek daži konflikti;
- apstājas, kad $f = 0$ vai beidzas laika limits.

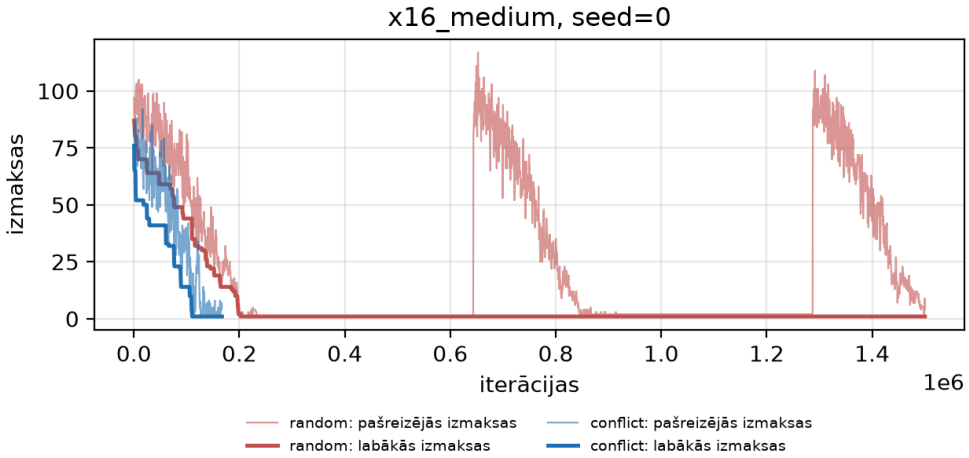
Parametrus izvēlējos, nedaudz paeksperimentējot ar 16×16 instanci (mēģināju $\alpha \in \{0.99, 0.995, 0.999\}$ un dažādus L). Atšķirības nebija pārāk lielas, un pilnīgi drošs, ka šie ir labākie, neesmu – visticamāk, ar rūpīgāku pielāgošanu var dabūt vēl kaut ko.

3. Testēšana

Testa instances ģenerēju pats (`generate_instances.py`): ar parastu pārlases (backtracking) algoritmu uztaisa pilnu X-Sudoku un tad izņem ciparus, kamēr risinājums joprojām ir viens vienīgs. Kopā 11 instances no 4×4 līdz 25×25 ar dažādu doto ciparu daudzumu. Optimālais risinājums katrai ir zināms (glabājas `*.solution.txt`), un optimālās izmaksas ir 0.

Katru instanci risināju ar abiem gājienu veidiem, katru 5 reizes ar dažādiem seed. Laika limits vienai palaišanai: 5 s (4×4), 20 s (9×9), 60 s (16×16), 120 s (25×25). Dators: AMD Ryzen 7 5700X, 8 palaišanas paralēli (katra savā procesā), tāpēc laiki ir aptuveni. Tabulā: *atr.* – cik no 5 palaišanām atrada optimumu, *laiks* – vidējais laiks veiksmīgajās palaišanās sekundēs, *izm.* – labākās atrastās izmaksas (min/vid/max), ja kāda palaišana optimumu neatrada.

			random			conflict		
instance	izmērs	doti	atr.	laiks	izm.	atr.	laiks	izm.
x4_easy	4×4	8 (50%)	5/5	0.00	0	5/5	0.00	0
x4_hard	4×4	4 (25%)	5/5	0.00	0	5/5	0.00	0
x9_easy	9×9	36 (44%)	5/5	0.18	0	5/5	0.02	0
x9_medium	9×9	28 (35%)	5/5	1.25	0	5/5	0.17	0
x9_hard	9×9	20 (25%)	1/5	6.87	0/0.8/1	1/5	19.57	0/0.8/1
x16_easy	16×16	160 (62%)	5/5	0.60	0	5/5	0.23	0
x16_medium	16×16	128 (50%)	5/5	18.99	0	5/5	6.31	0
x16_hard	16×16	100 (39%)	0/5	–	3/3.6/4	0/5	–	1/1.8/2
x25_easy	25×25	420 (67%)	5/5	1.50	0	5/5	0.81	0
x25_medium	25×25	360 (58%)	5/5	4.12	0	5/5	1.82	0
x25_hard	25×25	300 (48%)	0/5	–	16/17.2/19	1/5	32.77	0/5.6/11



Izmaksas atkarībā no iterācijām uz x16_medium. Asie lēcieni uz augšu ir reheat momenti.

Novērojumi.

- Ja doto ciparu ir pietiekami daudz, SA atrod optimumu visās 5 palaišanās un diezgan ātri, pat 25×25 laukumam ar 58% dotiem cipariem tas ir ap 2 sekundēm. Kur ir robeža, gan atkarīgs no izmēra: 9×9 ar 35% vēl atrisinās bez problēmām, bet 25×25 ar 48% jau lielākoties nē.
- Uz grūtajām instancēm algoritms ātri nonāk līdz dažiem konfliktiem (x9_hard un x16_hard izmaksas 1–2), bet pēdējos atrisināt ir grūti, jo tam bieži vajag izjaukt lielu daļu no jau "gandrīz pareizā" risinājuma. Reheat laikam palīdz (x9_hard un x25_hard pa reizei atrisinājās), bet ne vienmēr.
- *conflict* gājiens gandrīz visur ir labāks: jau uz 9×9 tas ir vairākas reizes ātrāks, un uz x25_hard atšķirība ir vislielākā (vidēji 5.6 pret 17.2). Uz 4×4 atšķirības nav, bet tur jau viss atrisinās uzreiz. Uz x9_hard abi varianti atrisināja tikai 1 no 5, tā ka tur neko īsti secināt nevar.
- Godīgi sakot, uz šī uzdevuma SA zaudē parastam backtracking ar MRV heuristiku – tas (tas pats, ko izmantoju instanču ģenerēšanai) jebkuru no šīm instancēm atrisina mazāk nekā 0.2 sekundēs. Tas laikam nav pārsteidzoši, jo Sudoku ir ļoti "stingri" ierobežots uzdevums, kur constraint propagation strādā ļoti labi, bet lokālajai meklēšanai ainava ir diezgan nelīdzena ar daudziem lokāliem minimumiem.
- Varbūt varētu uzlabot, ja pirms SA aizpildītu "acīmredzamās" šūnas (kurām ir tikai viens iespējamais cipars), vai arī ja izmaksu funkcijā kaut kā ņemtu vērā, cik daudz kandidātu katrai šūnai paliek. To gan vairs neizmēģināju.

4. Repozitorijs

Kods, testa instances un rezultāti (`results/results.csv`):

`https://github.com/kristiansrasins/x-sudoku_optimizer`

Palaišana: `python generate_instances.py` (instances jau ir repozitorijā), `python run_tests.py`, `python plot_convergence.py`.